Flask

Backend com Python

LTP3 + QP3 — Introdução a Web com Flask

Prof. Rafael Martins Alves — CEMIC 2026

Como funciona a Web?

Frontend

O que o usuário VÊ

- HTML (estrutura)
- CSS (estilo)
- JavaScript (interação)

HTTP

Backend

O que roda no SERVIDOR

- Python / Flask (lógica)
- Banco de dados
- APIs e rotas
- Validação de dados

O que é Flask?

Micro-framework

Leve e simples. Não obriga a usar banco de dados ou formulários complexos.

Python puro

Usa o Python que vocês já conhecem. Sem linguagem nova para aprender.

Usado no mercado

Netflix, Pinterest, LinkedIn usam Flask. Tecnologia profissional.

```
pip install flask
```

>_ Primeiro App: Hello World

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def inicio():
    return "Hello World!"

if __name__ == "__main__":
    app.run(debug=True)
```

Passo a passo

1. Importa o Flask
2. Cria a aplicação
3. @app.route define a URL
4. A função retorna o conteúdo
5. debug=True recarrega sozinho

```
python app.py
```

Rotas: URLs do seu App

```python
# Rota simples
@app.route("/")
def inicio():
    return "Página inicial"

# Rota com parâmetro
@app.route("/aluno/<nome>")
def aluno(nome):
    return f"Olá, {nome}!"

# Rota com cálculo
@app.route("/somar/<int:a>/<int:b>")
def somar(a, b):
    return f"{a} + {b} = {a + b}"
```

Templates HTML com Jinja2

app.py

```python
from flask import Flask,
     render_template, request

@app.route("/imc", methods=["GET","POST"])
def imc():
    if request.method == "POST":
        peso = float(request.form["peso"])
        alt = float(request.form["alt"])
        r = peso / alt ** 2
        return render_template(
            "imc.html", resultado=r)
    return render_template("imc.html")
```

templates/imc.html

```html
<form method="POST">
  <input name="peso"
    placeholder="Peso (kg)">
  <input name="alt"
    placeholder="Altura (m)">
  <button>Calcular</button>
</form>

{% if resultado %}
  <p>IMC: {{ resultado }}</p>
{% endif %}
```

Estrutura: meu_app/ > app.py + templates/ > imc.html — Flask procura .html em templates/

Estrutura de um Projeto Flask

```
meu-projeto/
  app.py
  requirements.txt
  templates/
    index.html
    resultado.html
  static/
    style.css
  tests/
    test_app.py
```

`app.py`	Arquivo principal com rotas e lógica
`requirements.txt`	Dependências (flask)
`templates/`	Arquivos HTML com Jinja2
`static/`	CSS, imagens, JavaScript
`tests/`	Testes automáticos (pytest)

Mini-Projeto Flask

Individual — Escolha 1 tema — Entrega via GitHub Classroom

1	**Calculadora de IMC**	Formulário peso/altura, calcula e classifica
2	**Conversor de Moedas**	Valor + moeda origem/destino, converte
3	**Lista de Tarefas**	Adicionar, listar e remover tarefas
4	**Cadastro de Alunos**	Nome + nota, listar aprovados/reprovados
5	**Quiz de Python**	Perguntas sobre Python, calcula pontuação

Requisitos: app.py funcional + pelo menos 1 template HTML + formulário POST + validação

Critérios de Avaliação

Critério	Descrição	Pontos
App roda	python app.py funciona sem erro	20
Rotas	Pelo menos 2 rotas funcionais	20
Template HTML	Usa render_template com Jinja2	20
Formulário	Formulário POST que processa dados	20
Validação	Valida entrada (try/except ou if/else)	20

Total: 100 pontos — Nota compartilhada LTP3 + QP3